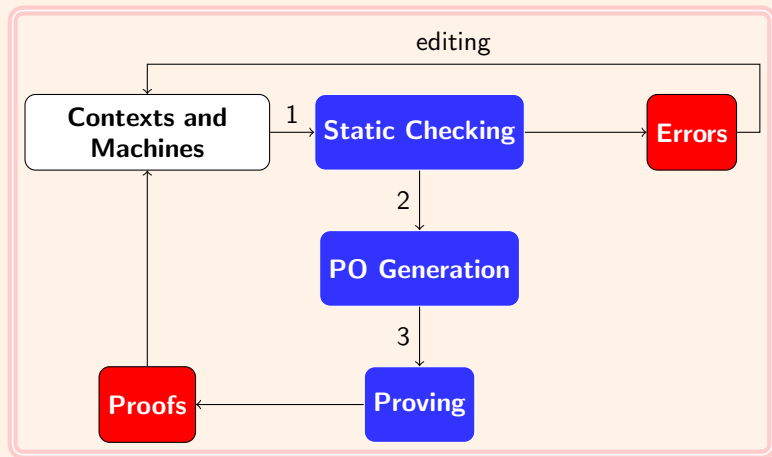


- 1 Overview of machines, contexts and proof obligations
- 2 Proof Obligations for Refinement
 - PO evt/grd/GRD
 - PO evt/act/SIM
 - PO evt/NAT
 - PO NAT
 - PO evt/VAR (arithmetic)
 - PO evt/VAR (set-theoretic)
 - PO evt/x/WFIS
- 3 Example of the factorial function refined into an algorithm
- 4 Transformations of EVENT B models

Current Section Summary

- 1 Overview of machines, contexts and proof obligations
- 2 Proof Obligations for Refinement
 - PO evt/grd/GRD
 - PO evt/act/SIM
 - PO evt/NAT
 - PO NAT
 - PO evt/VAR (arithmetic)
 - PO evt/VAR (set-theoretic)
 - PO evt/x/WFIS
- 3 Example of the factorial function refined into an algorithm
- 4 Transformations of EVENT B models

Analysis of the Event-B Models



MACHINE

m

REFINES

am

SEES

c

VARIABLES

u

INVARIANTS

$I(s, c, u)$

THEOREMS

$Q(s, c, u)$

VARIANT

$exp(s, c, u)$

EVENTS

INITIALIZATION

...

e

...

END

MACHINE

m

REFINES

am

SEES

c

VARIABLES

u

INVARIANTS

$I(s, c, u)$

THEOREMS

$Q(s, c, u)$

VARIANT

$exp(s, c, u)$

EVENTS

INITIALIZATION

...

e

...

END

- $\Gamma(m)$: environment for the machine m defined by the context c and it provides a list of seen axioms $Ax(s, c)$ and a list of seen theorems $Th(s, c)$ for the sets s and constants c .
- $\Gamma(m) \vdash \forall u. \text{INIT}(s, c, u) \Rightarrow I(s, c, u)$
- For each event e in E :
 $\Gamma(m) \vdash \forall u, u'. I(s, c, u) \wedge BA(e)(u, u') \Rightarrow I(u')$
- For each event e in E :
 $\Gamma(m) \vdash \forall u. I(s, c, u) \wedge GRD(e)(s, c, u) \Rightarrow \exists u'. BA(e)(u, u')$
- $\Gamma(m) \vdash \forall u. I(s, c, u) \Rightarrow Q(s, c, u)$
- Generated proof obligations are derived from those conditions.

Checking the well formation of Event-B expressions

- Event-B expressions are contexts, machines, properties, equations, set-theoretical expressions ...
- e is an Event-B expression and $\text{wd}(e)$ is a logical property expressing the well definition of e .
- $\text{wd}(1 = 2) \triangleq \text{wd}(1) \wedge \text{wd}(2)$
- $\text{wd}(a/b) \triangleq b \neq 0 \wedge \text{wd}(a) \wedge \text{wd}(b)$
- $\text{wd}(f(g)) \triangleq g \in \text{dom}(f) \wedge f \in A \leftrightarrow B$

Current Section Summary

- 1 Overview of machines, contexts and proof obligations
- 2 Proof Obligations for Refinement
 - PO evt/grd/GRD
 - PO evt/act/SIM
 - PO evt/NAT
 - PO NAT
 - PO evt/VAR (arithmetic)
 - PO evt/VAR (set-theoretic)
 - PO evt/x/WFIS
- 3 Example of the factorial function refined into an algorithm
- 4 Transformations of EVENT B models

Current Sub Section Summary

① Overview of machines, contexts and proof obligations

② Proof Obligations for Refinement

PO evt/grd/GRD

PO evt/act/SIM

PO evt/NAT

PO NAT

PO evt/VAR (arithmetic)

PO evt/VAR (set-theoretic)

PO evt/x/WFIS

③ Example of the factorial function refined into an algorithm

④ Transformations of
Telecom Nancy 2026-2027 Curr (Dominique Méry)

PO evt/grd/GRD

EVENT _{ae}

ANY x **WHERE**

$G(x, s, c, u)$

THEN

$u : |ABAP(x, s, c, u, u')$

END

EVENT _{ce}

REFINES

_{ae}

ANY y **WHERE**

$H(y, s, c, v)$

WITH

$x : W(x, y, s, c, v)$

THEN

$v : |CBAP(y, s, c, v, v')$

END

s

c

u, v

$Ax(s, c)$

$Th(s, c)$

$I(s, c, u)$

$J(s, c, u, v)$

$Q(s, c, u), R(s, c, u, v)$

ae, ce

x, y

$G(x, s, c, u)$

$H(y, s, c, v)$

$ABAP(x, s, c, u, u')$

$CBAP(x, s, c, u, u')$

$W(x, y, s, c, v)$

seen sets

seen constants

abstract and concrete variables

seen axioms

seen theorems

abstract invariants

concrete invariants

abstract and concrete theorems

abstract and concrete event names

event parameters

abstract event guard

concrete event guard

abstract event before-after predicate

concrete event before-after predicate

witness predicate

PO evt/grd/GRD

$Ax(s, c), Th(s, c), I(s, c, u), J(s, c, u, v), W(x, y, s, c, v), H(y, s, c, v), \vdash$
 $G(x, s, c, u, u')$

Current Sub Section Summary

① Overview of machines, contexts and proof obligations

② Proof Obligations for Refinement

PO evt/grd/GRD

PO evt/act/SIM

PO evt/NAT

PO NAT

PO evt/VAR (arithmetic)

PO evt/VAR (set-theoretic)

PO evt/x/WFIS

③ Example of the factorial function refined into an algorithm

④ Transformations of
Telecom Nancy 2026-2027 Curr (Dominique Méry)

EVENT _{ae}

ANY x **WHERE**

$G(x, s, c, u)$

THEN

$u : |ABAP(x, s, c, u, u')$

END

EVENT _{ce}

REFINES

ae

ANY y **WHERE**

$H(y, s, c, v)$

WITH

$x : WP(x, y, s, c, v)$

$u' : WV(y, u', s, c, v)$

THEN

$v : |CBAP(y, s, c, v, v')$

END

s

c

u, v

$Ax(s, c)$

$Th(s, c)$

$I(s, c, u)$

$J(s, c, u, v)$

$Q(s, c, u), R(s, c, u, v)$

ae, ce

x, y

$G(x, s, c, u)$

$H(y, s, c, v)$

$ABAP(x, s, c, u, u')$

$CBAP(x, s, c, u, u')$

$WP(x, y, s, c, v)$

$WV(y, u', s, c, v)$

seen sets

seen constants

abstract and concrete variables

seen axioms

seen theorems

abstract invariants

concrete invariants

abstract and concrete theorems

abstract and concrete event names

event parameters

abstract event guard

concrete event guard

abstract event before-after predicate

concrete event before-after predicate

witness parameter predicate

witness variable predicate

PO evt/act/SIM

$$\left(\begin{array}{l} Ax(s, c), Th(s, c), I(s, c, u), J(s, c, u, v) \\ WP(x, y, s, c, v), WV(y, u', s, c, v) \\ H(y, s, c, v), CBAP(y, s, c, v, v') \end{array} \right) \vdash ABAP(x, s, c, u, u')$$

Current Sub Section Summary

① Overview of machines, contexts and proof obligations

② Proof Obligations for Refinement

PO evt/grd/GRD

PO evt/act/SIM

PO evt/NAT

PO NAT

PO evt/VAR (arithmetic)

PO evt/VAR (set-theoretic)

PO evt/x/WFIS

③ Example of the factorial function refined into an algorithm

④ Transformations of
Telecom Nancy 2026-2027 Curr (Dominique Méry)

```
EVENT ae
  ANY x WHERE
     $G(x, s, c, u)$ 
  THEN
     $u : |BAP(x, s, c, u, u')$ 
  END
...
VARIANT
   $exp(s, c, u)$ 
```

s
 c
 u, v
 $Ax(s, c)$
 $Th(s, c)$
 $I(s, c, u)$
 $J(s, c, u, v)$
 $Q(s, c, u), R(s, c, u, v)$
evt, ce
 x
 $G(x, s, c, u)$
 $BAP(x, s, c, u, u')$
 $exp(s, c, u)$

seen sets
seen constants
abstract and concrete variables
seen axioms
seen theorems
abstract invariants
concrete invariants
abstract and concrete theorems
event name
event parameters
abstract event guard
event before-after predicate
arithmetic expression

PO evt/NAT

$Ax(s, c), Th(s, c), I(s, c, u), J(s, c, u, v), G(x, s, c, u) \vdash exp(s, c, u) \in \mathbb{N}$

Current Sub Section Summary

② Proof Obligations for Refinement

PO evt/grd/GRD

PO evt/act/SIM

PO evt/NAT

PO NAT

PO evt/VAR (arithmetic)

PO evt/VAR (set-theoretic)

PO evt/x/WFIS

- ## 4 Transformations of

```

EVENT ae
  ANY x WHERE
     $G(x, s, c, u)$ 
  THEN
     $u : |BAP(x, s, c, u, u')$ 
  END
  ...
VARIANT
     $exp(s, c, u)$ 
    
```

s
 c
 u, v
 $Ax(s, c)$
 $Th(s, c)$
 $I(s, c, u)$
 $J(s, c, u, v)$
 $Q(s, c, u), R(s, c, u, v)$
 evt, ce
 x
 $G(x, s, c, u)$
 $BAP(x, s, c, u, u')$
 $setexp(s, c, u)$

seen sets
seen constants
abstract and concrete variables
seen axioms
seen theorems
abstract invariants
concrete invariants
abstract and concrete theorems
event name
event parameters
abstract event guard
event before-after predicate
set expression

PO evt/NAT $Ax(s, c), Th(s, c), I(s, c, u), J(s, c, u, v), G(x, s, c, u) \vdash$
 $finite(setexp(s, c, u))$

Current Sub Section Summary

② Proof Obligations for Refinement

PO evt/grd/GRD

PO evt/act/SIM

PO evt/NAT

PO NAT

PO evt/VAR (arithmetic)

PO evt/VAR (set-theoretic)

PO evt/x/WFIS

- ## 4 Transformations of

```

EVENT ae
  ANY x WHERE
     $G(x, s, c, u)$ 
  THEN
     $u : |BAP(x, s, c, u, u')$ 
  END
...
VARIANT
   $exp(s, c, u)$ 
    
```

s
 c
 u, v
 $Ax(s, c)$
 $Th(s, c)$
 $I(s, c, u)$
 $J(s, c, u, v)$
 $Q(s, c, u), R(s, c, u, v)$
 evt, ce
 x
 $G(x, s, c, u)$
 $BAP(x, s, c, u, u')$
 $exp(s, c, u)$

seen sets
seen constants
abstract and concrete variables
seen axioms
seen theorems
abstract invariants
concrete invariants
abstract and concrete theorems
event name
event parameters
abstract event guard
event before-after predicate
arithmetic expression

PO evt/VAR

$Ax(s, c), Th(s, c), I(s, c, u), J(s, c, u, v), G(x, s, c, u), BAP(x, s, c, u, u') \vdash$
 $exp(s, c, u') < exp(s, c, u)$

Current Sub Section Summary

② Proof Obligations for Refinement

PO evt/grd/GRD

PO evt/act/SIM

PO evt/NAT

PO NAT

PO evt/VAR (arithmetic)

PO evt/VAR (set-theoretic)

PO evt/x/WFIS

- ## 4 Transformations of

```

EVENT ae
  ANY x WHERE
     $G(x, s, c, u)$ 
  THEN
     $u : \mid BAP(x, s, c, u, u')$ 
  END
...
VARIANT
   $setexp(s, c, u)$ 

```

s
 c
 u, v
 $Ax(s, c)$
 $Th(s, c)$
 $I(s, c, u)$
 $J(s, c, u, v)$
 $Q(s, c, u), R(s, c, u, v)$
 evt, ce
 x
 $G(x, s, c, u)$
 $BAP(x, s, c, u, u')$
 $setexp(s, c, u)$

seen sets
seen constants
abstract and concrete variables
seen axioms
seen theorems
abstract invariants
concrete invariants
abstract and concrete theorems
event name
event parameters
abstract event guard
event before-after predicate
set-theoretic expression

PO evt/VAR

$Ax(s, c), Th(s, c), I(s, c, u), J(s, c, u, v), G(x, s, c, u), BAP(x, s, c, u, u') \vdash$
 $setexp(s, c, u') \subset setexp(s, c, u)$

Current Sub Section Summary

② Proof Obligations for Refinement

PO evt/grd/GRD

PO evt/act/SIM

PO evt/NAT

PO NAT

PO evt/VAR (arithmetic)

PO evt/VAR (set-theoretic)

PO evt/x/WFIS

- ## 4 Transformations of

EVENT ae

ANY x **WHERE**

$G(x, s, c, u)$

THEN

$u : |ABAP(x, s, c, u, u')$

END

EVENT ce

REFINES

ae

ANY y **WHERE**

$H(y, s, c, v)$

WITH

$x : WP(x, y, s, c, v)$

$u' : WV(y, u', s, c, v)$

THEN

$v : |CBAP(y, s, c, v, v')$

END

s

c

u, v

$Ax(s, c)$

$Th(s, c)$

$I(s, c, u)$

$J(s, c, u, v)$

$Q(s, c, u), R(s, c, u, v)$

ae, ce

x, y

$G(x, s, c, u)$

$H(y, s, c, v)$

$ABAP(x, s, c, u, u')$

$CBAP(x, s, c, u, u')$

$WP(x, y, s, c, v)$

$WV(y, u', s, c, v)$

seen sets

seen constants

abstract and concrete variables

seen axioms

seen theorems

abstract invariants

concrete invariants

abstract and concrete theorems

abstract and concrete event names

event parameters

abstract event guard

concrete event guard

abstract event before-after predicate

concrete event before-after predicate

witness parameter predicate

witness variable predicate

PO evt/x/WFIS

$Ax(s, c), Th(s, c), I(s, c, u), J(s, c, u, v), H(y, s, c, v) \vdash$

$\exists x. WP(x, y, s, c, v)$

Current Section Summary

- 1 Overview of machines, contexts and proof obligations
- 2 Proof Obligations for Refinement
 - PO evt/grd/GRD
 - PO evt/act/SIM
 - PO evt/NAT
 - PO NAT
 - PO evt/VAR (arithmetic)
 - PO evt/VAR (set-theoretic)
 - PO evt/x/WFIS
- 3 Example of the factorial function refined into an algorithm
- 4 Transformations of EVENT B models

The factorial model

CONTEXT *functions*

CONSTANTS *factorial* $n0$ $r0$

AXIOMS

@req1 $n0 \in \mathbb{N}$

@req2 $r0 \in \mathbb{Z}$

@axm2 *factorial* $\in \mathbb{N} \leftrightarrow \mathbb{N}$

@axm3 $0 \mapsto 1 \in \textit{factorial}$

@axm4 $\forall a. \forall b. a \in \mathbb{N} \wedge b \in \mathbb{N} \wedge a \mapsto b \in \textit{factorial}$
 $\Rightarrow a + 1 \mapsto (a + 1) * b \in \textit{factorial}$

@axm5 $\forall f. f \in \mathbb{N} \leftrightarrow \mathbb{N} \wedge 0 \mapsto 1 \in f \wedge$
 $(\forall a, b. a \in \mathbb{N} \wedge b \in \mathbb{N} \wedge a \mapsto b \in f$
 $\Rightarrow a + 1 \mapsto (a + 1) * b \in f)$
 $\Rightarrow \textit{factorial} \subseteq f$

theorem @th1 *factorial* $\in \mathbb{N} \rightarrow \mathbb{N}$

theorem @th2 *factorial*(0) = 1

theorem @th3 $\forall u. u \in \mathbb{N} \wedge u \neq 0 \Rightarrow \textit{factorial}(u) = u * \textit{factorial}(u - 1)$

- The context defines the mathematical function *factorial* by axioms 2,3,4 and 5 and the properties th1, thh2 and th3 as theorems.
- $n0$ is a constant defined as a natural number.

The factorial pre/post specification

```
MACHINE
  prepostok
  SEES  functions
  VARIABLES  r ok n
  INVARIANTS
    @inv1  $r \in \mathbb{Z}$ 
    @inv2  $n \in \mathbb{Z}$ 
    @inv3  $ok \in \text{BOOL}$ 
    @inv4  $n = n0$ 
    @post  $ok = \text{TRUE} \Rightarrow r = \text{factorial}(n)$ 
  EVENTS
    EVENT  INITIALISATION
      then
        @act1  $r : \in \mathbb{Z}$ 
        @act2  $ok := \text{FALSE}$ 
        @act3  $n := n0$ 
      end
    EVENT  computing1
      where
        @grd1  $ok = \text{FALSE}$ 
      then
        @act1  $ok := \text{TRUE}$ 
        @act2  $r := \text{factorial}(n)$ 
      end
    end
```

- Three variables are used for expressing the end of the computation by assigning TRUE to *ok* , the mathematical value of *factorial(n)* to *r* and the variable *n* to *n0*.
- The event final/call the function *observes* the call of the *hidden* function computing the *factorial* function.
- The statement $r := \text{factorail}(n)$ expressses the relation between the mathematical value and the model variable *r*.
- Invariant *post* expresses the postcondition.

The factorial computing : defining the state

```
MACHINE computing
  REFINES prepostok
  SEES functions
  VARIABLES r fac k n ok
```

INVARIANTS

```
@inv1 fac ∈ ℕ ⇔ ℕ
@inv2 dom(fac) ⊆ 0..n
@inv3 dom(fac) ≠ ∅
@inv4 ∀ i. i ∈ dom(fac)
      ⇒ fac(i) = factorial(i)
@inv5 k ∈ dom(fac)
@inv6 dom(fac) = 0..k
@inv8 ok = TRUE ⇒ n ∈ dom(fac)
theorem @th1 n = n0
theorem @inv7 k ∈ 0..n0
VARIANT 0..n \ dom(fac)
```

- Previous variables n, r, ok are named in the same way and abstract variables are equal to concrete variables.
- Two new variables are introduced namey k and fac .
- fac is the partial copy of the function *factorial* until k .
- k defines the domain of fac .
- fac is a subset of *factorial* by *inv4*.

The clause VARIANT is new and expresses that the new machine or view has a new event which can not always take the control over the previous abstract events as *computing1* appreaing as *computing2* in the new machine.

The factorial computing : refining the computing

```
EVENT INITIALISATION
REFINES INITIALISATION
  then
    @act1  $r : \in \mathbb{Z}$ 
    @act2  $fac := \{0 \mapsto 1\}$ 
    @act3  $k := 0$ 
    @act4  $n := n0$ 
    @act5  $ok := FALSE$ 
  end
EVENT computing2
REFINES computing1
  where
    @grd1  $n \in dom(fac)$ 
    @grd2  $ok = FALSE$ 
  then
    @act1  $r := fac(n)$ 
    @act2  $ok := TRUE$ 
  end
```

- The two events are refined and are made more concrete by adding the two new variables.
- The event computing2 is concretizing the guard by the guard $n \in dom(fac)$.
- The computation is completed, when one can get the value corresponding to $factorial(n)$ which is stored in fac by the event step2.
- The concrete guard of the event computing2 is stronger than the abstract guard of the event computing1.

The concrete action of the event computing2 is simulating the abstract event of the event computing1 (computing2/act2/SIM).

The factorial computing : progressing for computing

```

convergent EVENT step2
  where
    @grd13  $n \notin \text{dom}(fac)$ 
    @grd14  $ok = FALSE$ 
  then
    @act11  $fac(k+1) := (k+1) * fac(k)$ 
    @act1  $k := k+1$ 
  end

```

- The event step2 is new and refines skip.
- The status is set to convergent and means that its occurrence decreases the value of the variant.
- It helps to converge.

The PO steps2/VAR expresses the decreasing of the variant :

$$0 .. n \setminus \text{dom}(fac \triangleleft \{ k+1 \mapsto (k+1) * fac(k) \}) \subset 0 .. n \setminus \text{dom}(fac)$$

Intermediate summary for the refinement process

- The current machines expresses the computing process of the variable *fac* as well as an index *k*.
- The computation of factorial(*n*) does not require the storing of intermediate values.
- *fac* is a *model* variable and we are now introducing the last required value for the computation.

The factorial pre-algorithmic machine

```
MACHINE prealgo
  REFINES computing
  SEES functions

VARIABLES  r fac k n ok ffac

INVARIANTS
  @inv1 ffac  $\in \mathbb{Z}$ 
  @inv2 ffac = fac(k)
  theorem @inv3 dom(fac) = 0..k
  theorem @inv4  $\forall i. i \in \text{dom}(\text{fac})$ 
     $\Rightarrow \text{fac}(i) = \text{factorial}(i)$ 
  @inv5 ok = TRUE  $\Rightarrow k = n$ 

VARIANT  n - k
```

- A new variable is introduced namely *ffac* and its role is to keep the last computed value of *factorial*.
- *inv2* is a key invariant stating the relation between *ffac* and *fac*.
- The invariant *inv5* is relating *ok* and *k*.

The factorial pre-algorithmic machine

```

EVENT INITIALISATION
  then
    @act1  $r \in \mathbb{Z}$ 
    @act2  $fac := \{0 \mapsto 1\}$ 
    @act3  $k := 0$ 
    @act4  $n := n_0$ 
    @act5  $ok := FALSE$ 
    @act6  $ffac := 1$ 
  end
EVENT computing3
  REFINES computing2
    where
      @grd1  $k = n$ 
      @grd2  $ok = FALSE$ 
    then
      @act1  $r := ffac$ 
      @act2  $ok := TRUE$ 
    end

```

- The event `computing3` is refining an event with a guard difficult to evaluate
 $n \in \text{dom}(fac)$ by strengthening the guard into the following form $k = n$.
- The action of `computing3` is simple too and `ffac` contains the correct value to assign.

The factorial pre-algorithmic machine

```

EVENT step3 REFINES step2
  where
    @grd1  $k \neq n$ 
    @grd2  $ok = FALSE$ 
    then
      @act1  $fac(k+1) := (k+1) * fac(k)$ 
      @act2  $k := k+1$ 
      @act3  $ffac := (k+1) * ffac$ 
    end

```

- The same transformation of the event `step2` into `step3` by using the condition $k \neq n$ instead of $n \notin \text{dom}(fac)$.
- The action $ffac := (k + 1) * ffac$ is added and simulates the inductive definition.

- The guards are made simpler and are easier to compute.
- The variable *ffac* is simulating the inductive step.
- The variable *fac* can be *hidden* and remains a model variable but is not implemented.

The factorial algorithmic machine

```

MACHINE algo
  REFINES prealgo
  SEES functions

VARIABLES  r k n ffac ok

INVARIANTS
  theorem @th1 ffac = fac(k)
  theorem @th2 k ∈ 0..n
  theorem @th3 n = n0
  theorem @th4 ok = TRUE ⇒ k = n
  theorem @th5 ok = TRUE ⇒ r = factorial(n)

```

- The new model is hidden the variable *fac* which is a model variable.
- Properties of variables are inherited as theorems from the previous steps.
- The theorem th5 retains the expression of the partial correctness derived from the process.

The factorial algorithmic machine

```
EVENT INITIALISATION
```

```
  then
```

```
    @act1  $r : \in \mathbb{Z}$ 
```

```
    @act3  $k := 0$ 
```

```
    @act4  $n := n0$ 
```

```
    @act6  $ffac := 1$ 
```

```
    @act7  $ok := FALSE$ 
```

```
  end
```

```
EVENT computing4
```

```
  REFINES computing3
```

```
    where
```

```
      @grd1  $k = n$ 
```

```
      @grd2  $ok = FALSE$ 
```

```
      then
```

```
        @act1  $r := ffac$ 
```

```
        @act2  $ok := TRUE$ 
```

```
    end
```

```
EVENT step4 REFINES step3
```

```
  where
```

```
    @grd1  $k \neq n$ 
```

```
    @grd2  $ok = FALSE$ 
```

```
    then
```

```
      @act2  $k := k + 1$ 
```

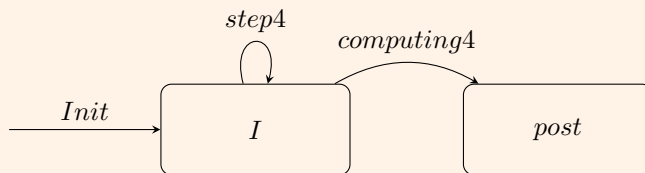
```
      @act3  $ffac := (k + 1) * ffac$ 
```

```
    end
```

```
  end
```

**The events are no more containing the variable *fac*.
Guards are simpler to transform into an algorithmic statement.**

Summary of the refinement-based development

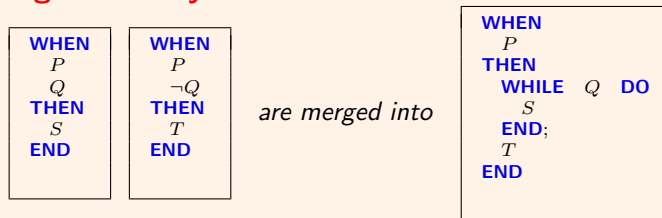


- The last machine is simple and has no more problematic expressions.
- The diagram expresses that *step4* is observed until the event *computing4* is observed.
- The diagram leads to an iteration.

Current Section Summary

- 1 Overview of machines, contexts and proof obligations
- 2 Proof Obligations for Refinement
 - PO evt/grd/GRD
 - PO evt/act/SIM
 - PO evt/NAT
 - PO NAT
 - PO evt/VAR (arithmetic)
 - PO evt/VAR (set-theoretic)
 - PO evt/x/WFIS
- 3 Example of the factorial function refined into an algorithm
- 4 Transformations of EVENT B models

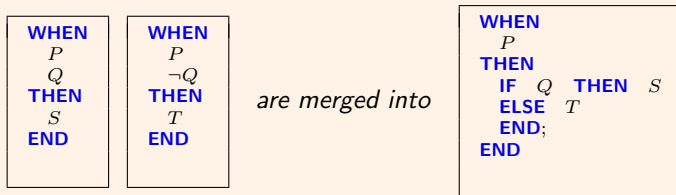
Merging Rules for transforming Event B models into algorithms by J.-R. Abrial



Side Conditions :

- P must be invariant under S .
- The first event must have been introduced at one refinement step below the second one.
- Special Case : If P is missing the resulting "event" has no guard

Merging Rules for transforming Event B models



Side Conditions :

- The disjunctive negation of the previous side conditions
- Special Case : If P is missing the resulting "event" has no guard

Deriving an algorithm

```
VARIABLES  $r, k, n, ffac, ok$   
requires  $n0 \in \mathbb{N}$   
ensures  $r = factorial(n0)$   
begin  
   $k := 0; n := n0; ffac := 1; ok := FALSE;$   
  while  $k \neq n$  do  
     $k := k + 1;$   
     $ffac := k * ffac;$   
  od;  
   $r := ffac;$   
   $ok := TRUE;$   
end
```